

Article

Edge Detection in Colored Images Using Parallel CNNs and Social Spider Optimization

Jiahao Zhang ^{1,2} , Wei Wang ^{1,2,*} and Jianfei Wang ^{3,*}

¹ Co-Innovation Center of Efficient Processing and Utilization of Forest Resource, Nanjing Forestry University, Nanjing 210037, China; zhangjiahao@njfu.edu.cn

² Colleges of Furnishings and Industrial Design, Nanjing Forestry University, Nanjing 210037, China

³ School of Software, Tongji University, Shanghai 201800, China

* Correspondence: wangwei1219@njfu.edu.cn (W.W.); 2031546@tongji.edu.cn (J.W.)

Abstract: Edge detection is a crucial issue in computer vision, with convolutional neural networks (CNNs) being a key component in various systems for detecting edges within images, offering numerous practical implementations. This paper introduces a hybrid approach for edge detection in color images using an enhanced holistically led edge detection (HED) structure. The method consists of two primary phases: edge approximation based on parallel convolutional neural networks (PCNNs) and edge enhancement based on social spider optimization (SSO). The first phase uses two parallel CNN models to preliminarily approximate image edges. The first model uses edge-detected images from the Otsu-Canny operator, while the second model accepts RGB color images as input. The output of the proposed PCNN model is compared with pairwise combination of color layers in the input image. In the second phase, the SSO algorithm is used to optimize the edge detection result, modifying edges in the approximate image to minimize differences with the resulting color layer combinations. The experimental results demonstrate that our proposed method achieved a precision of 0.95. Furthermore, the peak signal-to-noise ratio (PSNR) and structural similarity index measure (SSIM) values stand at 20.39 and 0.83, respectively. The high PSNR value of our method signifies superior output quality, showing reduced contrast and noise compared to the ground truth image. Similarly, the SSIM value indicates that the method's edge structure surpasses that of the ground truth image, further affirming its superiority over other methods.



Citation: Zhang, J.; Wang, W.; Wang, J. Edge Detection in Colored Images Using Parallel CNNs and Social Spider Optimization. *Electronics* **2024**, *13*, 3540. <https://doi.org/10.3390/electronics13173540>

Academic Editor: Gemma Piella

Received: 2 April 2024

Revised: 20 August 2024

Accepted: 4 September 2024

Published: 6 September 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: edge detection; social spider optimization; convolutional neural network; deep learning

1. Introduction

In the fields of computer vision and image processing, such as image compression [1], corner detection [2], and image segmentation [3], edge detection has been developed for many years. In images, edges denote lines or curves when there is a sudden change in color or grayscale [4]. With the advancement of deep learning techniques, edges are still helpful in certain applications and serve as local features to identify information in images, even if the majority of tasks can now be completed by an end-to-end training process. Edges are used in medical image analysis to determine the extent of cancers [5]. The edge-based segmentation approach is a significant branch in the field of image segmentation [6]. As color images and videos become more widely used, color edge detection is becoming more and more common. It is common knowledge that gray edge detectors can identify only 90% of the edges in color images. Still, in certain computer vision applications, the residual 10% of edge pixels often are significant [6]. Additionally, it was discovered that the capacity to recognize edges is significantly impacted by chromatic information [7,8]. Consequently, rather than thinking of color edge detection as just a straightforward extension of gray edge detection, color edge detectors must be developed [9]. Traditional color edge detectors, learning-based detectors, and deep learning-based detectors are the three categories into which color edge detectors can be divided.

The majority of conventional edge detection methods rely on an image's gradients or directional derivatives. Based on the two gradients of an intensity image, the gray Canny detector [10] is extended to color images [11] by integrating the gradients of three components into a color image's Jacobian matrix. In a similar vein, the directional derivatives of three components are combined to create an anisotropic directional derivative (ANDD) matrix, extending the gray ANDD detector [12] to color images [13]. The robust color morphological gradient-median mean (RCMG-MM) edge detector [14] is dependent on the morphological gradients of an image, whereas the color anisotropic morphological directional derivative (AMDD) detector [7] is produced by integrating the direction derivatives with weighted median filter for images with impulsive noise. Additionally, for color edge identification, the principal component analysis-gradient operator based on hue difference (PCA-GOHD) edge detector [15] computes the gradients in an image's principal component. The color edge drawing (ColorED) detector [16] was invented recently by combining edge drawing with gradients [9]. The recent deep learning methods have been introduced as an efficient way to identify the edges of images and are widely used. However, the output of these methods often leads to a significant number of false positives, meaning that neighboring pixels to the edges that are not part of the edge are usually identified by the convolutional neural network models as part of the edge. This results in the formation of excessive points in the edge-detected image, creating relatively thicker edges compared to the actual edges of the image.

To address this issue, our proposed method presents a parallel model of CNNs that simultaneously takes two different modes of the image as input. The first input is an RGB structure of the image that decomposes the color layers of the image in five stages and identifies the edge points with different levels of detail through different depths of the convolutional neural network. Ultimately, it seeks to achieve a more accurate result of the image edges by combining these. However, to mitigate the issue of excessive false positives in the edge detection output, we have added a parallel model to this model, which takes the edge-detected image through the Otsu-Canny method as input and attempts to mitigate this issue by combining the extracted features from this image through a model CNN with less depth and combining its results with those of the features extracted through the first CNN model. Thus, the parallel structure that is proposed here uses a combination of edge approximation and an RGB image as its input to overcome this issue. In this study, a new innovation has been introduced that adds a parallel model to the method. This parallel model takes the image resulting from applying the Otsu-Canny method to detect edges as input and then attempts to address this issue by combining the features extracted from this image using a shallower CNN model and integrating the results with the features extracted from the original CNN model. In summary, this parallel structure using the combination of edge approximation and an RGB image for its input resolves the existing issues in edge detection. Additionally, the output provided by the CNN model is contrast-based and may exhibit slight differences compared to our expected output. To mitigate these differences, we have employed optimization techniques in our proposed method. Despite our image being an RGB image, we have utilized combinations of pairwise color layers to obtain a more stable output, and by comparing the edge-detected image with these combinations, we aim to stabilize the identified edge conditions as much as possible to achieve better results. Therefore, this additional aspect is an innovation discussed in our proposed method, and studies have shown that each of these techniques can be effective in improving edge detection efficiency. The following is a summary of our principal contributions.

- Presenting a hybrid technique for image edge detection using a combination of deep learning techniques and social spider algorithm optimization.
- Introducing a parallel model of convolutional neural networks for a more accurate approximation of edges in color images.
- Using an optimization technique based on combinations of color layers to improve the edges of detected images through a deep learning technique.

This paper continues as follows: In the Section 2, we examine several related studies. The materials and technique are covered in the Section 3. The Section 4 examines the findings, while the Section 5 summarizes them.

2. Related Works

Wen et al. [17] proposed an edge detector that utilizes feature re-extraction in a deep convolutional neural network. This detector comprises three modules: a backbone, a side-output, and feature fusion. The detector's generalization ability has been validated on the BSDS500 dataset and its cross-distribution performance on the NYUDv2 dataset. Freezing the backbone network accelerated training without compromising the effectiveness.

Qu et al. [18] introduced a new edge detector that utilizes a visual cross-fusion (VCF) network to extract features from a multi-level hierarchy through parameter dimension reduction and cross-fusion of fully connected layers. The algorithm improved the accuracy and FPS by 5.66% and 10% on the Berkeley Segmentation Data Set and NYUD V2 datasets.

Wibisono and Hang [19] presented a traditional DNN architecture for edge detection based on classical concepts. The system achieved good edge quality with a simpler model. Experiments showed that the traditional inspired network (TIN1) produced good results with a small model, while TIN2 exhibited higher accuracy.

Wang et al. [20] introduced the richer category-aware semantic edge detection network (R-CASENet), a multi-category edge detection network that utilized the feature expression capabilities of convolutional neural networks for accurate feature extraction and classification. It also introduced an edge refinement network structure for precise one-pixel width edges, improving computational efficiency.

Fang et al. [21] suggested a feature decoder-based approach that employs a feature decoder network (FDN) to extract additional information from constrained CNN features. The system used CNN features to distinguish between edge and non-edge pixels, understanding the link between low-level edge hierarchies and high-level semantic hierarchies. The algorithm outperformed cutting-edge algorithms in experimental results.

Le and Duan [22] offered REDN, a recursive encoder-decoder network with skip-connections, for edge identification in natural pictures. It combined a Recursive Neural Network and an Encoder-Decoder architecture to allow for iterative edge refinement. REDN considerably improved edge detection in conventional evaluation criteria such as optimal dataset scale and average precision, based on extensive testing.

Soria et al. [23] developed a lightweight dense convolutional (LDC) neural network for edge detection based on two cutting-edge models. It required less than 4% of parameters and produced thin edge maps, outperforming lightweight models and heavy architectures.

Wibisono and Hang [24] described fast inference network for edge detection (FINED), a lightweight deep learning-based edge detection system. The system obtained good accuracy while minimizing model complexity. The method also included a training assistant idea, which reduced model complexity while retaining accuracy. Experiments demonstrated that FINED outperformed contemporary edge detectors at the same model size.

Liu et al. [25] offered an Edge Attention Network (EdgeAtNet) that uses the richer convolutional features (RCF) architecture. It included global view attention blocks for low-level features and local focus attention for high-level ones. EdgeAtNet outperformed other approaches on various benchmarks, including the BSDS500, NYUDv2, and BIPED datasets, with an ODS F1-measure of 0.825.

Al-Amaren et al. [26] provided a new VGG16-based DCNN technique that employs residual learning to achieve high performance while remaining simple. Experiments with diverse datasets demonstrated that the proposed network outperformed all other VGG16-based edge detection algorithms.

Ye et al. [27] explored self-training for edge detection using large-scale unlabeled image datasets. A self-supervised framework was designed with multilayer regularization and self-teaching, imposing consistency regularization and L0-smoothing for edge prediction. The network was trained with pseudo labels and refined iteratively. This approach achieved

a balance of precision and recall, boosting performance over supervised methods and improving the original edge detector performance after self-training and fine-tuning.

Elharrouss et al. [28] presented CHRNet, a cascaded, high-resolution network, to address these issues. During training, the approach used batch normalization for homogeneous regions while maintaining high edge resolution. In terms of performance metrics and image quality, the technique performed better than planned edge detection networks.

Wang et al. [29] developed an edge detection network that combines an enhanced Holistically-Nested edge detection (HED) network with adaptive Canny fusion. VGG16 served as the network's backbone, integrating feature extraction with extended and empty convolutions. The model responded better to complicated scenes and produced superior edge detection results at several scales. However, certain complicated scenes still showed over-detection and feature missing, requiring additional investigation.

Soria et al. [30] introduced a new edge dataset and a unique architecture known as the dense extreme inception network for edge detection (DexiNed), which could be trained from scratch without pre-learned weights. DexiNed outperformed other algorithms in the presented dataset and performed well on other datasets without requiring fine-tuning. It also provided sharper, finer edges.

Poma et al. [31] recommended a deep learning-based edge detector modeled after Holistically-Nested Edge Detection (HED) and Xception networks. It produced narrow edge maps adequate for human vision and could be applied to any edge detection task without training or fine-tuning. A large dataset with thoroughly labeled edges was created for training and comparison. The evaluations revealed improvements in the F1-measure of ODS and OIS.

3. Proposed Method

This section introduces a novel hybrid approach for edge detection in color images. The proposed method leverages an enhanced Holistically-nested Edge Detection (HED) architecture [32] for initial edge approximation, followed by optimization techniques to refine the detected edges. The architecture of the proposed method, as illustrated in Figure 1, can be broken down into two primary phases:

- (1) Edge approximation based on parallel CNNs (PCNN);
- (2) Edge enhancement based on social spider optimization (SSO).

In the first phase of the proposed method, two convolutional neural network CNN models are used in parallel for the preliminary approximation of image edges. Each of these CNN models is designed based on the HED structure, and they collaborate to provide a suitable approximation of the image edges by sharing detected edge information.

The first CNN model is fed with edge-detected images produced by the Otsu-Canny operator [33]. The Otsu-Canny operator is a hybrid method that combines the Otsu thresholding technique with the Canny edge detector to produce a binary image highlighting potential edges.

The second CNN model accepts RGB color images as input. RGB images consist of three color layers (red, green, and blue), and this model analyzes these layers to detect edges based on the color information.

The output from the PCNN model forms an edge approximation matrix, denoted as E_A . This matrix represents the approximated edges detected by the parallel CNNs. Next, this edge approximation matrix E_A is compared with binary combinations of the color layers of the input image. The goal here is to refine the edge detection results by optimizing the approximation using the social spider optimization (SSO) algorithm.

The following is an explanation of the variables:

E_A : Edge approximation matrix generated by the PCNN model. This matrix encapsulates the preliminary detected edges from both CNN models.

R_R , R_G , and R_B : These represent the binary representations of the red, green, and blue color layers of the input image, respectively.

R_C : The combination of the binary color layers R_R , R_G , and R_B that is used as a reference to evaluate and optimize the edge detection.

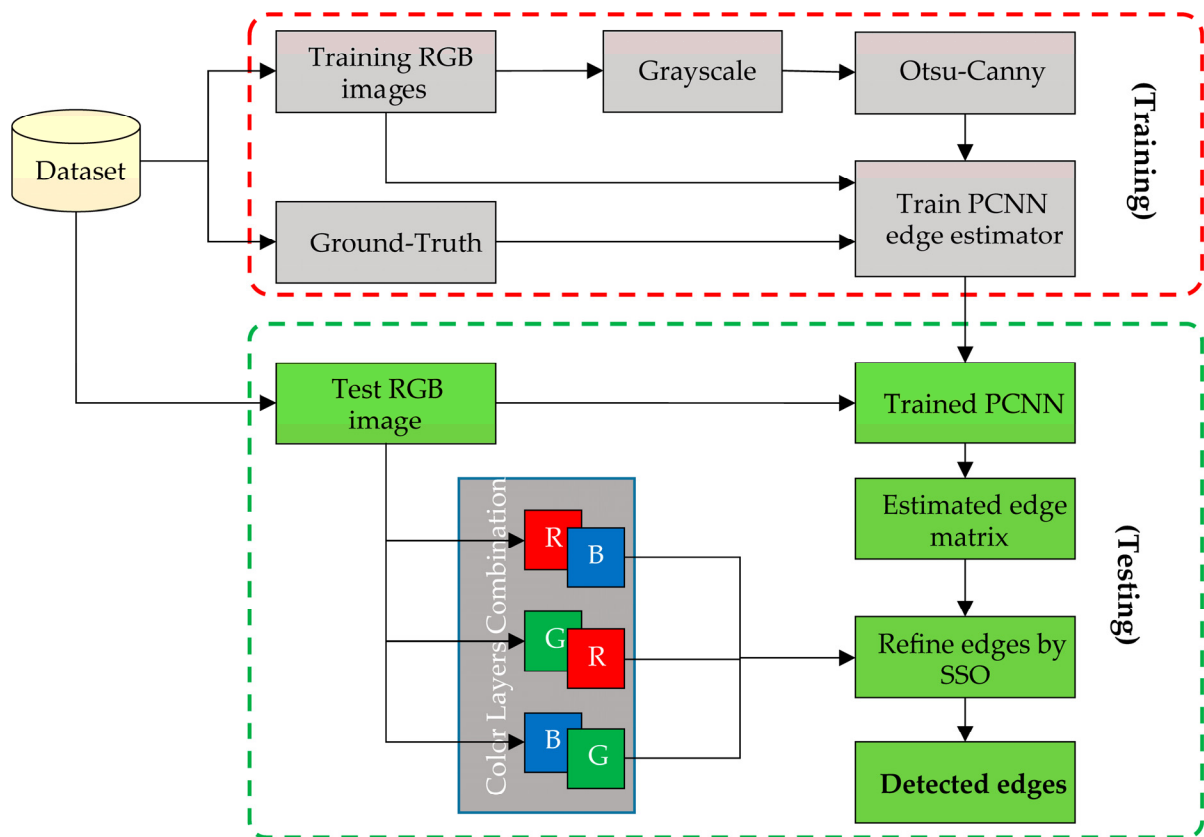


Figure 1. Schematic representation of the proposed model for edge detection.

$\text{Dist}(E_A, R_C)$: This function measures the difference between the edge approximation matrix E_A and the combined color layer reference R_C .

In this step, the SSO algorithm strives to minimize the difference $\text{Dist}(E_A, R_C)$, modifying the edges in E_A so that they align more closely with the edges formed by the binary combinations of the color layers. This process aims to enhance the accuracy and clarity of the detected edges, leading to a more refined edge detection result.

3.1. Edge Approximation Based on Parallel CNNs (PCNN)

The first step of the proposed method employs a PCNN model for the initial approximation of image edges. This model comprises two CNNs, each aiming to create a more accurate edge approximation. They achieve this by sharing edge detection information extracted from different representations of the input image. Each CNN model in this structure is designed based on the HED architecture. The basic structure of the HED architecture for edge detection in images is depicted in Figure 2. As per this architecture, the basic HED model hierarchically extracts features related to the edge of the input image using convolutional layers. The output of each convolutional stage is compressed using a max function in a pooling layer, resulting in a reduction in the dimensions of the feature map matrix and a decrease in image details. Consequently, in deeper convolutional layers, edge details are reduced, and only the primary edges of the image are extracted. After repeating this process over five stages, five edge detection images are obtained. These images are then appended together through a concatenation layer. By combining the edge features in these partial images, the output of the HED model is created.

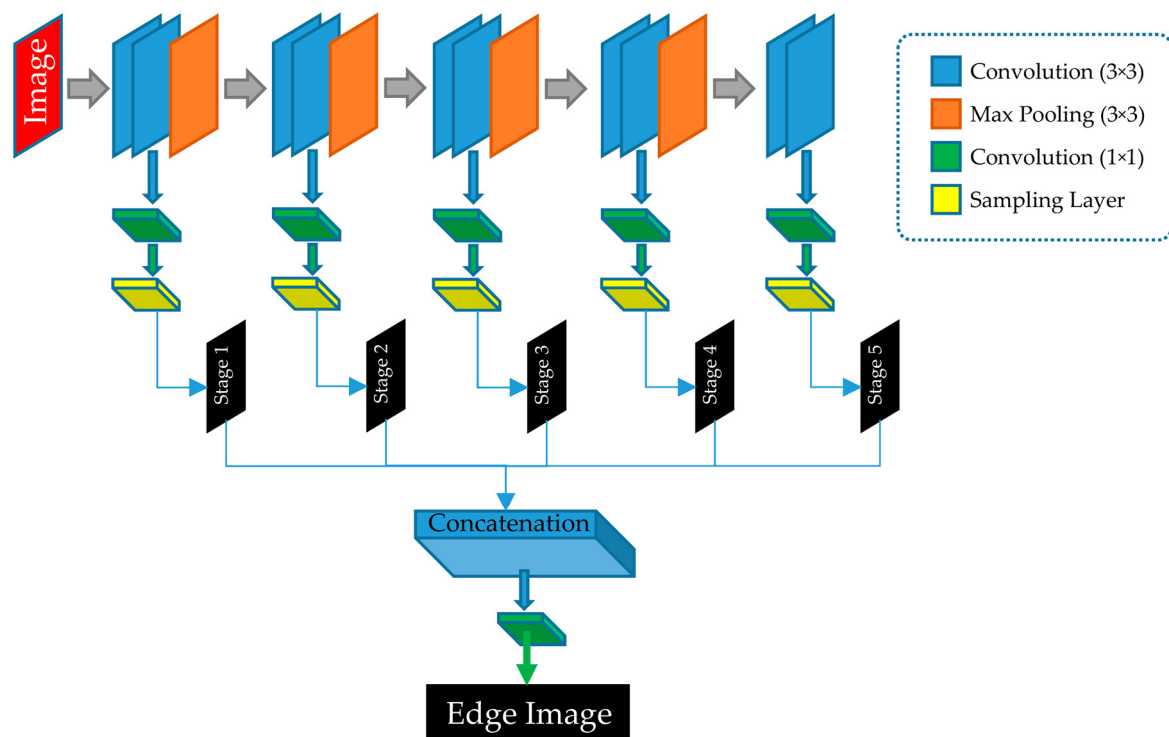


Figure 2. Basic HED network architecture for image edge detection.

One of the reasons for using the HED architecture as the basic structure of the CNN models in the proposed method is the non-use of fully connected layers, which has two advantages. Firstly, this largely eliminates the model's performance dependency on the dimensions of the image, allowing images of different dimensions to be edge-detected by this model. Secondly, the removal of fully connected layers in the CNN architecture significantly reduces the number of learnable parameters, thereby reducing the complexity of the model. Each CNN model used in the proposed PCNN architecture strives to obtain the edges of the images through the hierarchical extraction of features. However, in the proposed PCNN model, one of the CNN models accepts the color image as input, while the second CNN model is fed with the edge-detected images by the Otsu-Canny operator. Using the Otsu-Canny edge approximation as one of the inputs of the proposed PCNN model offers two advantages. Firstly, the application of Otsu-Canny edges, which are somewhat close to the ground truth edges of the image, can effectively reduce the complexity of the CNN model, thus enabling edge approximation in fewer hierarchies. On the other hand, one of the disadvantages of deep models in the edge detection of images is the inability to collect global information of edge-detected images, due to the limitations of the convolution operator. This strategy can effectively reduce the structural complexities of the input image and address this challenge. The proposed PCNN architecture for edge approximation in RGB images is depicted in Figure 3.

In the proposed PCNN architecture, the CNN1 model extracts edge features through three convolution components, while this process for the RGB version of the image is performed through CNN2 with a depth of 5. The edge detection input matrices of Otsu-Canny, compared to the RGB version of the image, have negligible structural features. For this reason, all the edge-related features in it can be extracted through three levels.

After extracting the edge matrices at different levels for the two models, CNN1 and CNN2, these matrices are appended together using a Depthcat layer based on depth to obtain a matrix with a depth of 8. Finally, a convolution layer with dimensions of 1×1 is used to merge these features and form the final edge matrix.

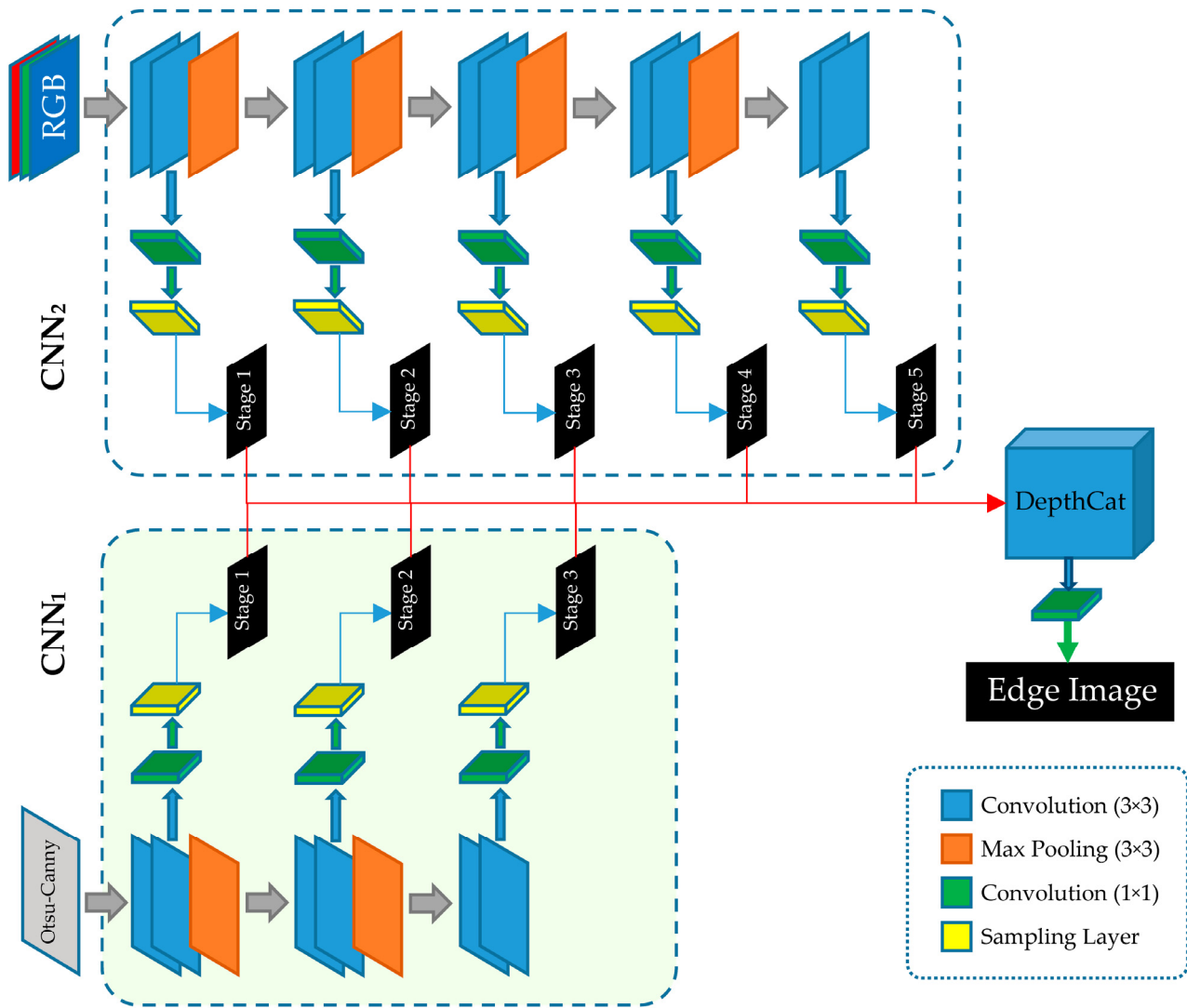


Figure 3. Proposed PCNN architecture for edge approximation in RGB images.

Similar to the HED model, the proposed PCNN structure also converts the image edge detection problem into a binary classification problem, where each output pixel falls into one of the target categories: “edge” or “non-edge”. For this purpose, to calculate the loss in the output of each level of the CNN models, the weighted sum of the losses of all outputs is used.

$$Loss_S(X, x) = \sum_{i=1}^n w_i \times C_s^i(X, x^i) \quad (1)$$

where w_i represents the weight of the loss function for the i th level output of the model. The loss function $C_s^i(X, x^i)$ is defined as the cross-entropy loss, which measures the difference between the predicted output and the ground truth at the i th level. Specifically, $C_s^i(X, x^i)$ processes every pixel in the input image X and compares it to the corresponding pixel in the i th level output (x^i) during training. The response value of each level of the edge detection model, denoted as R_s^i , is computed using logistic regression, which outputs the probability of a pixel being an edge. In this case, the loss function resulting from the merging of the side outputs of the PCNN model is formulated as follows:

$$Loss_F(X, x, h) = Dist(R, R_F) \quad (2)$$

where $Dist(R, R_F)$ represents the difference between the ground truth label R and the generated output R_F . This distance is calculated using the cross-entropy function, which

quantifies the dissimilarity between the predicted output R_F and the ground truth R . Specifically, it measures the average number of bits needed to identify the true labels from the predicted probability distributions. In this model, R_F can be calculated as follows:

$$R_F = \sigma \sum_{i=1}^n h_i \times R_s^i \quad (3)$$

where h represents the method of merging for the output of each level, and n describes the number of levels producing side outputs in the PCNN model. Thus, based on the total cost of merging the outputs of each level of the model and also the cost of the output of each level individually, the parameters of the convolution model can be optimized. For this purpose, the gradient descent strategy is used iteratively based on the following equation:

$$(X, x, h)^* = \operatorname{argmin}(Loss_S(X, x) + Loss_F(X, x, h)) \quad (4)$$

The output of the trained PCNN model for each input sample is a binary matrix that represents the approximation of the image edges. This matrix serves as the input for the second phase of the proposed method.

3.2. Edge Enhancement Based on SSO

The SSO algorithm is chosen over others for its efficiency, precision, adaptability, and robustness in optimizing edge pixels with respect to color matrices. After obtaining the edge approximation matrix, SSO is employed to enhance the quality of the edges in the image. Consider the edge approximation matrix (the result of the previous stage) as shown in Figure 4a. In this stage, the SSO algorithm attempts to match each of the pixels located on the edge with the brightness intensity values of the three matrices L1, L2, and L3 (L1 from red and green, L2 from red and blue, and L3 from green and blue). These matrices are the result of binary combinations of the color layers of the input image. In this context, each of the edge pixels is considered as an optimization variable that can be moved a maximum of one pixel using the solution vector. In Figure 4a, each of the edge pixels is specified with variables v1 to v8. Therefore, for this sample image, the length of the solution vector is equal to 8.

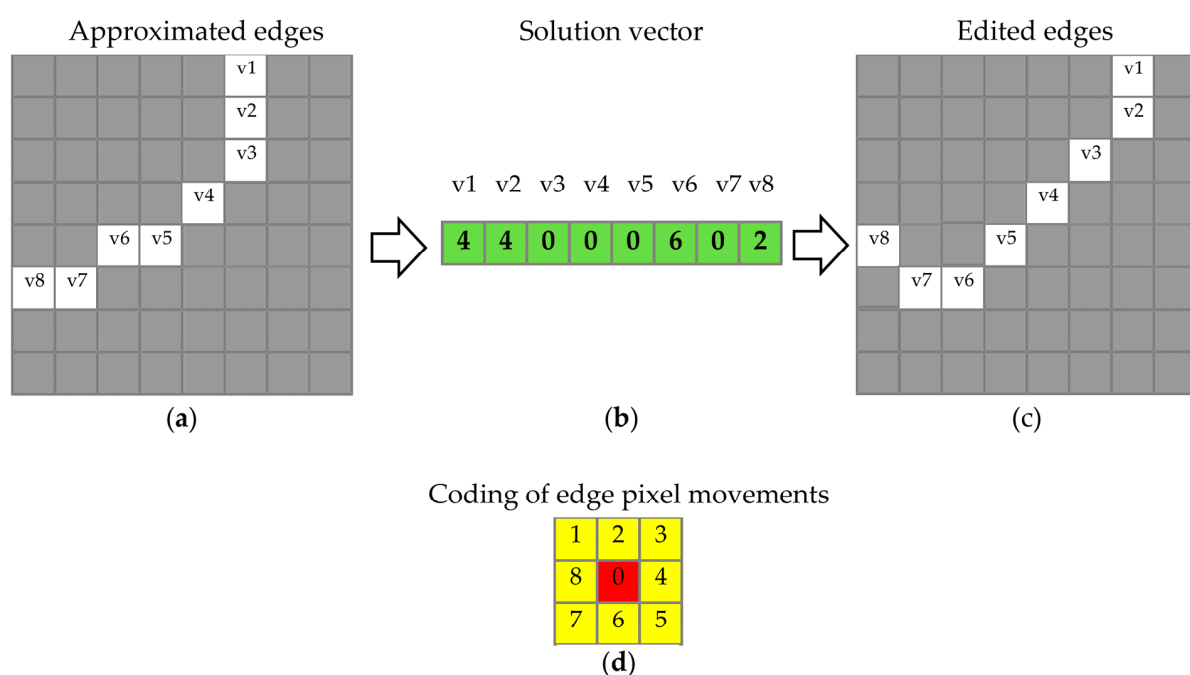


Figure 4. An example of edge improvement process by SSO algorithm: (a) Approximated edges (b) Solution vector (c) Edited edges (d) Coding of edge pixel movements.

An example of the solution vector is illustrated in Figure 4b. Each component of this solution vector corresponds to one of the optimization variables (edge pixels v1 to v8), and the number present in each position of the solution vector determines the movement of that edge pixel. The encoding method of the solution vector for each pixel is provided in Figure 4d. According to this figure, the number 0 signifies no movement of the edge pixel, and numbers 1 to 8 indicate movement in one of the surrounding directions with a radius of 1. By applying the solution vector in Figure 4b to the edge matrix of Figure 4a, Figure 4c is obtained. In this figure, pixels v1, v2, v6, and v8 are moved according to the pattern determined in the hypothetical solution vector.

In this stage, SSO is employed to enhance the quality of the edges in the initial approximation image. The length of the solution vector of this algorithm is equal to the number of pixels located on the edge in the initial approximation image. Each variable in this vector represents the position of an edge pixel and can have values from 0 to 8.

The fitness evaluation of each solution vector is based on comparing the values of edge pixels in binary combinations of color layers of the original image. For this purpose, each binary combination of the color layers of the image is first converted into a matrix. This results in three matrices, L1, L2, and L3, derived from the combination of RG, GB, and BR layers, respectively.

$$L_c = \frac{1}{2} \times (I_1 + I_2), \quad I_1, I_2 \in \{R, G, B\}, \quad I_1 \neq I_2 \quad (5)$$

In the above equation, I_1, I_2 describe two different color layers of the RGB image, and L_c represents the uniform combination of these two layers. The fitness criterion is defined as the average standard deviation of the edge pixel values in layers L1, L2, and L3. The objective of the SSO algorithm is to minimize the following criterion:

$$Fitness(S) = \frac{1}{1 + \frac{1}{3|S|} \sum_{i=1}^3 \sum_{j=1}^{|S|} std\left(L_i^j \cup N\left(L_i^j\right)\right)} \quad (6)$$

where L_i^j represents the value of the j th edge pixel in layer L_i ($i = 1, 2, 3$), and $N\left(L_i^j\right)$ represents the set of values of the neighbors of that pixel (located within a radius of one). Additionally, $std(.)$ describes the standard deviation function. Finally, $|S|$ indicates the number of optimization variables or the number of pixels that are part of the edge.

The SSO algorithm at this stage adjusts the position of the edge pixels in such a way that the fitness criterion is minimized. The optimal solution vector of this algorithm is applied to the edge approximation matrix, and the final image is obtained.

The steps of edge optimization by the SSO algorithm are as follows:

(Step 1) The initial population of solution vectors (spiders) is randomly determined based on the search bounds.

(Step 2) The fitness of each solution vector is calculated based on Equation (6), and then the weight value of each spider is evaluated based on Equation (7) [34]:

$$w_i = \frac{fitness_i - worst}{best - worst} \quad (7)$$

where $fitness_i$ represents the calculated fitness for the i th solution, and worst and best respectively represent the worst and best fitness values in the current population.

(Step 3) Calculate the vibration amount of each spider agent based on the following equation [34]:

$$V_{(i,j)} = w_j e^{d_{i,j}^2} \quad (8)$$

where w_j represents the weight assigned to spider j through Equation (7), and $d_{i,j}$ represents the Euclidean distance between spiders i and j . The Euclidean distance is used because it

effectively measures the straight-line distance between two points in the multi-dimensional space where the spiders are located.

The exponential function is employed in Formula (8) to model the sensitivity of a spider's response to vibrations. As the distance between two spiders increases, the influence of the vibration diminishes exponentially, reflecting a rapid decrease in impact as the spiders move further apart. This helps in simulating the natural behavior where closer interactions are more influential, enhancing the ability of the algorithm to exploit local optima while exploring the search space.

(Step 4) Update the position of the female spiders based on the current position and received vibration as follows [34]:

$$f_i(k+1) = \begin{cases} f_i(k) + \alpha \cdot V_{I,n} \cdot (s_n - f_i(k)) + \beta \cdot V_{i,b} \cdot (s_b - f_i(k)) + \delta \cdot (\text{rand} - 0.5) & \text{with } P_f \\ f_i(k) - \alpha \cdot V_{I,n} \cdot (s_n - f_i(k)) - \beta \cdot V_{i,b} \cdot (s_b - f_i(k)) + \delta \cdot (\text{rand} - 0.5) & \text{with } 1 - P_f \end{cases} \quad (9)$$

where the parameters α , β , and δ are random numbers in the range [0,1]. Additionally, s_n and s_b represent the position of the neighboring spider and the best spider, respectively. $f_i(k)$ represents the current position of spider I , and $f_i(k+1)$ denotes its updated position. The notation $V_{I,n}$ refers to the vibration value for spider I relative to its neighboring spider n .

Explanation of P_f : P_f stands for the probability that a female spider will follow the vibration cues from a neighboring spider as opposed to moving in a random direction. This probability controls the likelihood of the spider's movement being influenced by its neighbors, helping balance the trade-off between exploration and exploitation in the optimization process.

(Step 5) Update the position of the male spiders based on the current position and received vibration as follows [34]:

$$m_i(k+1) = \begin{cases} m_i(k) + \alpha \cdot V_{i,f} \cdot (s_f - m_i(k)) + \delta \cdot (\text{rand} - 0.5) & \text{if } m_i(k) \text{ is dominant} \\ f_i(k) - \alpha \cdot \left(\frac{\sum_{j \in ND} m_j(k) \cdot w_j}{\sum_{j \in ND} w_j} - m_i(k) \right) & \text{if } m_i(k) \text{ is nondominant} \end{cases} \quad (10)$$

where s_f represents the position of the nearest female spider.

Explanation of Dominant $m_i(k)$: In Formula (10), $m_i(k)$ is considered "dominant" if male spider i has a higher fitness value than other male spiders in its neighborhood. A dominant spider is more likely to influence the optimization process by attracting others towards its position, representing a leader-like role within the swarm.

(Step 6) Apply the combination and rescue operator among the spider population and increase the counter of the number of algorithm iterations by one unit.

(Step 7) If the fitness reaches zero or the number of algorithm iterations is equal to the threshold G , proceed to the next step; otherwise, repeat the algorithm from step 2.

(Step 8) Apply the solution with the least fitness to the image matrix and return the result as the edge detection result.

4. Research Findings

In this section, we evaluate the results of the proposed method implementation and performance assessment on the BSDS500 database [35] using MATLAB 2018b software. We conducted this assessment using a 10-fold cross-validation technique. Furthermore, we provided explanations about the data structure and the performance evaluation of the methods, followed by the presentation of the implementation results.

Data and Evaluation Criteria

The proposed method has been implemented and its performance in edge detection has been evaluated using samples from the BSDS500 database [35]. This database comprises three sets of images: training (200 samples), validation (100 samples), and test (200 samples). These images have been collected specifically for segmentation and edge detection tasks. All images are stored in the RGB color system and have varying dimensions. Each sample

in the BSDS500 database has five ground truth edge detections. During the experiments, the proposed model was trained based on the first set in this database.

If we consider edge detection to be a classification issue, then each pixel can be viewed as a sample. In this situation, an image's pixels are separated into two categories: the background region (negative class), and the edge pixels (positive class). By comparing the proposed method's edge detection results to the ground truth image for each sample, we can assign each pixel to one of four states.

- True Positive (TP): The number of actual positive pixels that are correctly identified by the model.
- False Negative (FN): The number of actual positive pixels that the model incorrectly categorizes as negative.
- False Positive (FP): The number of actual negative pixels that the model erroneously categorizes as positive.
- True Negative (TN): The number of actual negative pixels that the model accurately identifies as negative.

Precision is a performance metric that quantifies the number of accurate positive forecasts. It evaluates the accuracy of the minority class by computing the ratio of correctly predicted positive instances to the total number of predicted positive instances. This computation is utilized to estimate precision.

$$Precision = \frac{TP}{TP + FP} \quad (11)$$

Recall measures the number of correct positive predictions out of all available positive predictions. Unlike precision, which only considers correct positive predictions among all positive predictions, recall also includes the positive predictions that were missed. The recall is calculated using a specific formula.

$$Recall = \frac{TP}{FN + TP} \quad (12)$$

A good classifier should have both precision and recall equal to one, which means both FP and FN should be zero. As a result, we require a statistic that considers both precision and recall. The $F1$ -measure, which takes into account precision and recall, is defined as follows:

$$F - Measure = \frac{2 \times Precision \times recall}{Precision + recall} \quad (13)$$

The mean square error (MSE) is a measuring metric used to determine the accuracy of a model or forecast. The formula for MSE is as follows.

$$MSE = \left(\frac{1}{n} \right) \times \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (14)$$

where Y_i represents the actual values, and $\hat{Y}_i$ represents the predicted values by the model or system.

The $PSNR$ formula is used in image and video processing to assess the quality of multimedia files. This equation determines the ratio of the signal energy to the noise in a signal and is defined as

$$PSNR = 10 \times \log_{10} \frac{(255^2)}{MSE} \quad (15)$$

where 255 represents the maximum possible value for 8-bit image pixels, and MSE indicates the difference between the actual and the predicted values.

The *SSIM* equation is a metric that determines the structural similarity of two photographs. This metric is based on three major criteria: structural similarity, brightness, and contrast. The *SSIM* is defined as follows:

$$SSIM(X, Y) = (2 \times \mu_x \times \mu_y + c1) \times \frac{2 \times \sigma_{xy} + c2}{\mu_x^2 + \mu_y^2 + c1} \times (\sigma_x^2 + \sigma_y^2 + c2) \quad (16)$$

where X and Y represent the two images. μ_x and μ_y denote the mean of each image. σ_x and σ_y represent the variance of each image. σ_{xy} indicates the correlation between X and Y . $c1$ and $c2$ are small constants to prevent division by zero in case the denominators are very small.

We used the BSDS500 database to compare the output of the proposed method with real data. This set includes images with five different edges for each input image, based on which images are identified. Additionally, we checked the proposed method in three different cases. The first mode is related to the full implementation of the proposed method, which is explained in the third section, to identify edge images in a two-phase manner. The difference between the proposed method and its second mode proposed (CNNRGB) lies in the fact that in the second mode, we solely utilize (CNNRGB) for detecting the edges of the image. Thus, the purpose of comparing these two modes is to evaluate the effectiveness of the parallel model employed for the proposed method. The mode proposed (without SSO) refers to using the first-phase model solely for edge detection without any edge enhancement operations through the SSO algorithm; in other words, in this mode, we entirely disregard the second phase. We additionally compared the proposed method to three other works, namely, Wang et al. [29], Soria et al. [30], and Poma et al. [31].

Figure 5 shows the changes in the fit of the SSO algorithm to improve the edges of the image, which, based on the fit evaluation function we defined, shows how effective this algorithm was in improving the edges of the image. This output shows that our proposed method is somewhat successful, and in 100 cycles, it is able to reach its own optimal solution, and the population is also downwardly directed towards global solutions. In other words, the algorithm has tried to direct the totality of responses to the global optimal response and has tried to find more appropriate responses in relation to it.

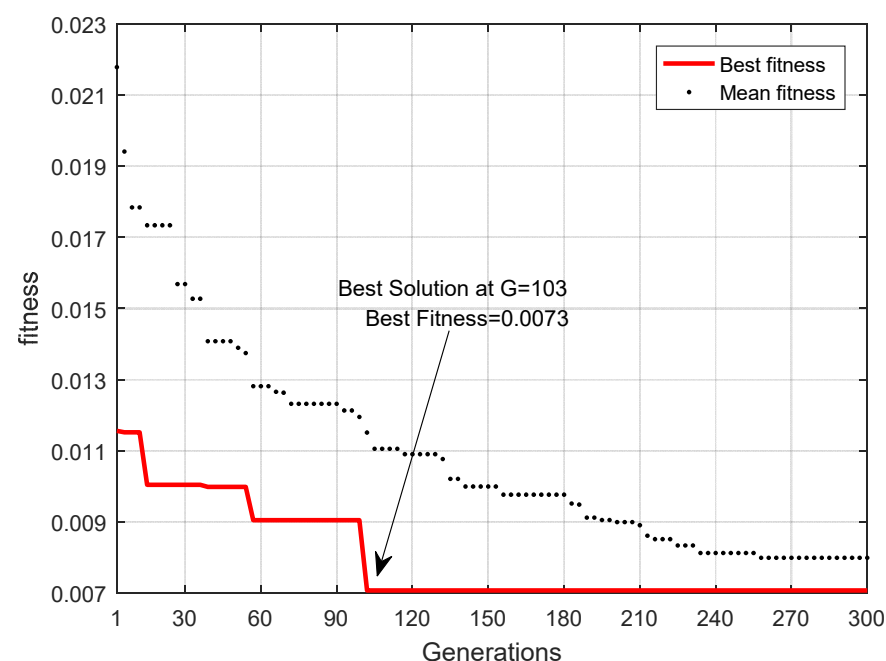


Figure 5. Variations in the average fitness and optimal fitness discovered for the optimization of the edges of a sample image.

In the examples presented in Figure 6, we see images from the BSDS500 database. In the second column, there are ground truth images taken from set one, and next to them, we have checked the output of the proposed method and three well-known methods (Sobel, Canny, and Prewitt) for image edge detection. These results show that the simultaneous use of both phases of the proposed PCNN and SSO method provides the best results for image edge detection. Comparing the mode of the proposed PCNN method with the CNN_{RGB} mode also shows that the use of the parallel model of neural networks yields a significant improvement over the use of CNN_{RGB} and reduces the false positive rate as much as possible.



Figure 6. Examples of edge-detected images using the proposed method, compared with other methods, sourced from the BSDS500 database.

Figure 7 shows the edge detection quality. Therefore, the results show that our proposed method (PCNN) mode has performed better than the other two models that were compared, and by increasing the precision, it effectively increases the recall. In other words, we were able to generate edge detection patterns in such a way that more pixels corresponding to edge regions are detected by the proposed method. This increase is visible based on the increase in recall, and the increase in precision in this method also shows that a higher proportion of pixels identified as edges are actually correct than with the compared models. In general, the F1-measure also shows that our proposed method has performed better than the compared models in terms of precision and recall, and this superiority applies to different modes of the proposed method and the compared models.

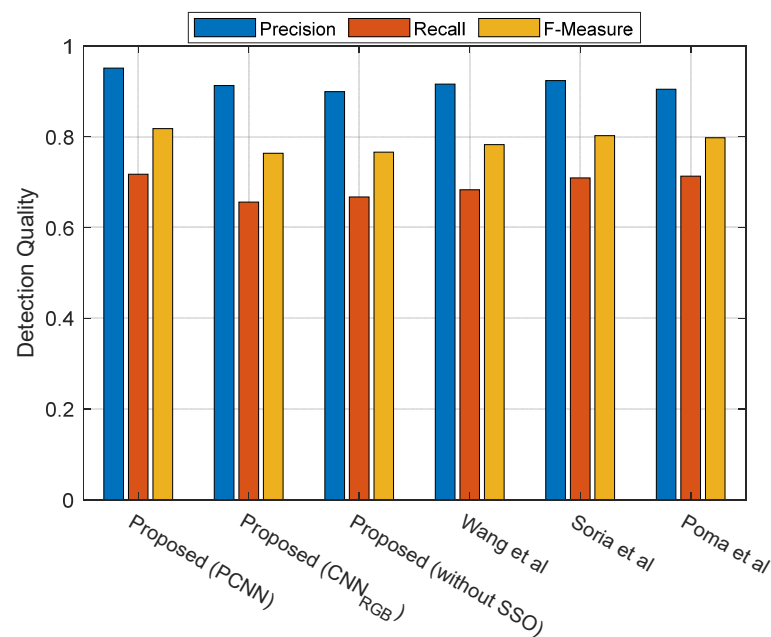


Figure 7. Efficiency of various methods in edge detection applied to BSDS500 color images, evaluated based on precision, recall, and F1-measure criteria [23,29,31].

In Figure 8, we have plotted the precision and recall values as a curve to show how the precision value is based on different recall thresholds. These results show that the proposed method, for different recall threshold values, is able to obtain higher precision values. In other words, the area under the curve for the proposed method shows an increase of 7% compared to the compared methods such as proposed (CNN_{RGB}). Moreover, compared to the other methods, the method of Wang et al. [29] is the closest method to the proposed method, but the proposed method has been able to increase by 6%.

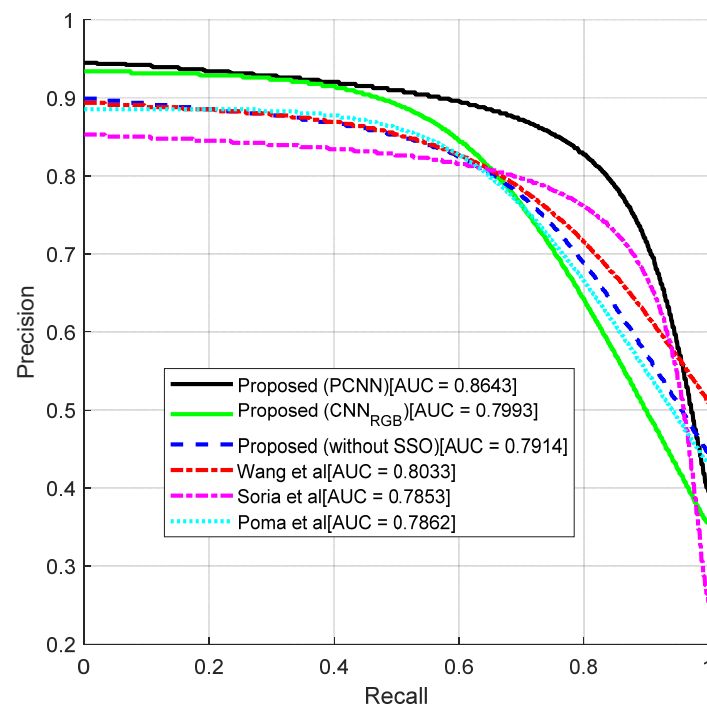


Figure 8. Plot of precision versus recall for various methods [23,29,31].

In Figure 9, we compared the performance of different methods based on the criteria of MSE, PSNR, and SSIM. A lower MSE value indicates better performance, which our proposed method has fulfilled by generating output with minimal difference compared to the ground truth image. This means that the actual edge image has been better estimated, and this reduction is consistent across the compared methods. Furthermore, higher values of PSNR and SSIM are preferable. A higher PSNR value indicates better alignment of the output generated by the proposed method with the ground truth image and better overall quality, meaning less discrepancy and noise in the image. Moreover, a higher SSIM value indicates a greater resemblance between the proposed method and the ground truth image in terms of edge structure. Overall, all these results demonstrate that the proposed method outperforms the compared methods.

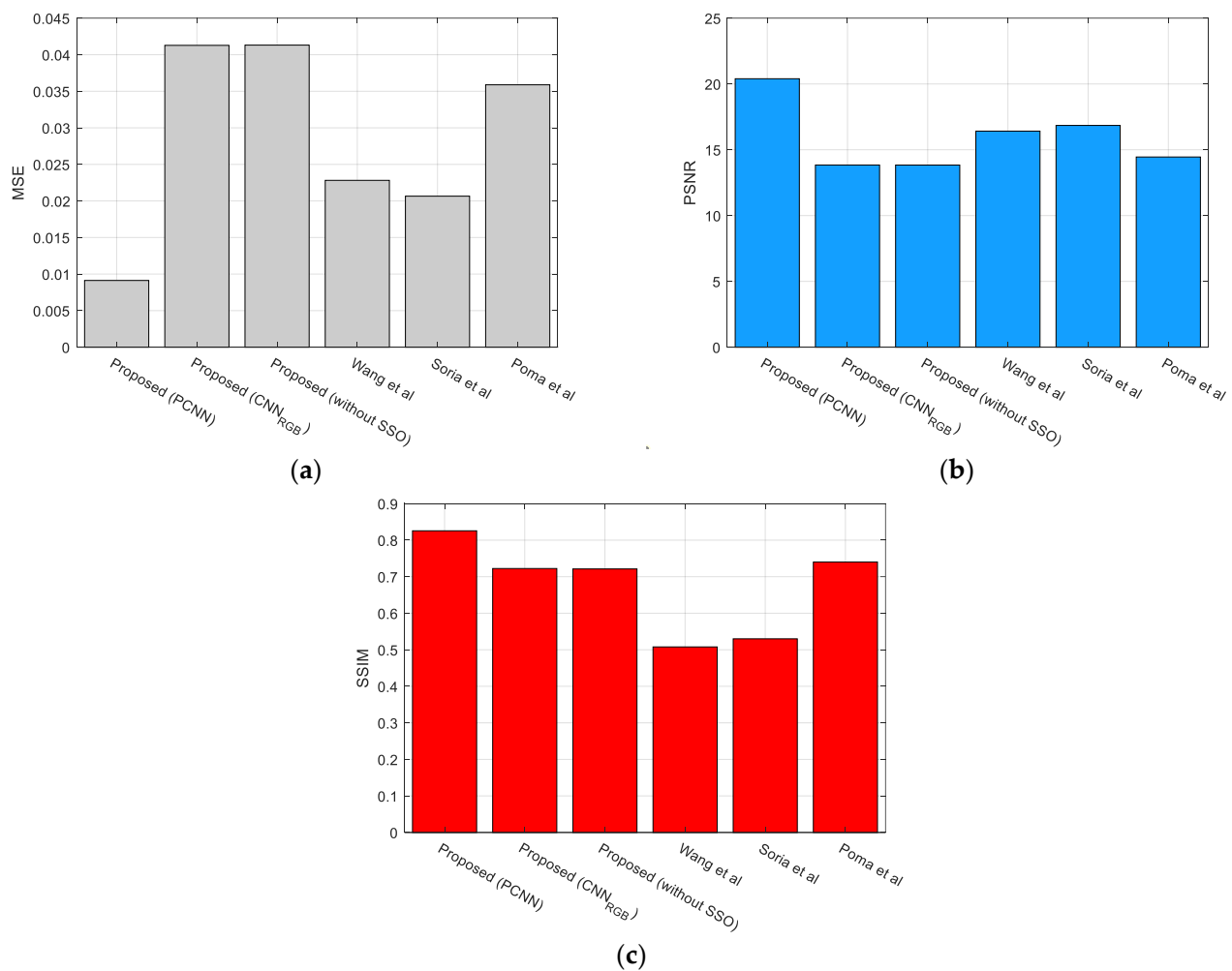


Figure 9. Performance comparison of various methods, evaluated by comparing the edge detection results with the background edge image using (a) MSE, (b) PSNR, and (c) SSIM criteria [23,29,31].

Table 1 presents metrics such as the MSE, PSNR, SSIM, precision, recall, and F1-measure. From this table, it can be inferred that the proposed (PCNN) method has the lowest MSE (0.0091), the highest PSNR (20.3974), and the highest SSIM (0.8256), indicating its superior performance compared to the other methods. On the other hand, Wang et al.'s [29] method exhibits the highest precision value (0.9159) with average values for the other metrics.

Table 1. Numerical values obtained from experiments.

Methods	F1-Measure	Recall	Precision	SSIM	PSNR	MSE	Time (s)
Proposed (PCNN)	0.8176	0.717	0.951	0.8256	20.3974	0.0091	3.3155
Proposed (CNN _{RGB})	0.7633	0.6559	0.9128	0.7224	13.8424	0.0413	2.0568
Proposed (without SSO)	0.766	0.6671	0.8992	0.7212	13.839	0.0413	3.3056
Wang et al. [29]	0.7825	0.6829	0.9159	0.5078	16.4174	0.0228	0.0015
Soria et al. [23]	0.8023	0.7091	0.9237	0.53	16.85	0.0207	2.0235
Poma et al. [31]	0.7975	0.713	0.9047	0.7401	14.4506	0.0359	1.5856

5. Conclusions

This paper introduces a hybrid approach for edge detection in color images using an enhanced HED structure. The method consists of two phases: edge approximation based on parallel convolutional neural networks (PCNN) and edge enhancement based on SSO. In the first phase, the PCNN model uses two parallel CNNs to approximate image edges, with one utilizing edge-detected images from the Otsu-Canny operator and the other accepting RGB color images as input. In the second phase, the SSO algorithm optimizes the edge detection result by modifying the edges to minimize differences with the resulting color layer combinations. The limitations of our proposed method, including the increased processing time compared to other methods, arise from the implementation of a two-stage phase. Firstly, the use of parallel CNN models can lead to increased processing time. Secondly, the second phase, which involves the optimization of image edges, increases the complexity of the problem and requires more time to enhance the image edges. In future work, we aim to incorporate more efficient computational methods to improve the processing time in our method, particularly compared to the initial version presented. The experimental results demonstrate the superiority of our method compared to other methods, with a precision of 0.95, a PSNR of 20.39, and an SSIM measurement of 0.83.

One of the limitations of the proposed method is its higher computation complexity which is the result of utilizing optimization steps. The need for further optimization of the proposed model for real-time implementation is a research topic for our future works. Furthermore, utilizing additional techniques to enhance the contrast of the images when converting them to grayscale is another research topic that can be addressed in future works.

Author Contributions: W.W. and J.Z. analyzed the data and wrote this manuscript. W.W. gave valuable suggestions on the experiments and manuscript. J.Z. and J.W. modified the manuscript in detail. All authors have read and agreed to the published version of the manuscript.

Funding: Art Project of National Social Science Foundation, National Social Science Office Project Number: 2023BG01252 “Research on Rural Landscape Ecological Design of Yangtze River Delta under the Background of Yangtze River Protection”.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Mainberger, M.; Bruhn, A.; Weickert, J.; Forchhammer, S. Edge-based compression of cartoon-like images with homogeneous diffusion. *Pattern Recognit.* **2011**, *44*, 1859–1873. [[CrossRef](#)]
2. Zhang, W.C.; Shui, P.L. Contour-based corner detection via angle difference of principal directions of anisotropic Gaussian directional derivatives. *Pattern Recognit.* **2015**, *48*, 2785–2797. [[CrossRef](#)]
3. Zhi, X.H.; Shen, H.B. Saliency driven region-edge-based top down level set evolution reveals the asynchronous focus in image segmentation. *Pattern Recognit.* **2018**, *80*, 241–255. [[CrossRef](#)]
4. Akbari, A.S.; Soraghan, J.J. Fuzzy-based multiscale edge detection. *Electron. Lett.* **2003**, *39*, 1.

5. Chen, H.; Qi, X.; Yu, L.; Dou, Q.; Qin, J.; Heng, P.A. DCAN: Deep contour-aware networks for object instance segmentation from histology images. *Med. Image Anal.* **2017**, *36*, 135–146. [\[CrossRef\]](#)
6. Liu, C.; Liu, W.; Xing, W. A weighted edge-based level set method based on multi-local statistical information for noisy image segmentation. *J. Vis. Commun. Image Represent.* **2019**, *59*, 89–107. [\[CrossRef\]](#)
7. Li, O.; Shui, P.L. Noise-robust color edge detection using anisotropic morphological directional derivative matrix. *Signal Process.* **2019**, *165*, 90–103. [\[CrossRef\]](#)
8. Mallat, S.; Hwang, W.L. Singularity detection and processing with wavelets. *IEEE Trans. Inf. Theory* **1992**, *38*, 617–643. [\[CrossRef\]](#)
9. Li, O.; Shui, P.L. Color edge detection by learning classification network with anisotropic directional derivative matrices. *Pattern Recognit.* **2021**, *118*, 108004. [\[CrossRef\]](#)
10. Akbari, A.S.; Zadeh, P.B.; Behringer, R. Iris Segmentation Using a Non-Decimated Wavelet Transform. In Proceedings of the 2nd IET International Conference on Intelligent Signal Processing 2015 (ISP), London, UK, 1–2 December 2015.
11. Koschan, A.; Abidi, M. Detection and classification of edges in color images. *IEEE Signal Process. Mag.* **2005**, *22*, 64–73. [\[CrossRef\]](#)
12. Zhang, W.; Zhao, Y.; Breckon, T.P.; Chen, L. Noise robust image edge detection based upon the automatic anisotropic Gaussian kernels. *Pattern Recognit.* **2017**, *63*, 193–205. [\[CrossRef\]](#)
13. Wang, F.P.; Shui, P.L. Noise-robust color edge detector using gradient matrix and anisotropic Gaussian directional derivative matrix. *Pattern Recognit.* **2016**, *52*, 346–357. [\[CrossRef\]](#)
14. Nezhadarya, E.; Ward, R.K. A new scheme for robust gradient vector estimation in color images. *IEEE Trans. Image Process.* **2011**, *20*, 2211–2220. [\[CrossRef\]](#) [\[PubMed\]](#)
15. Lei, T.; Fan, Y.; Wang, Y. Colour edge detection based on the fusion of hue component and principal component analysis. *IET Image Process.* **2014**, *8*, 44–55. [\[CrossRef\]](#)
16. Akinlar, C.; Topal, C. Colored: Color edge and segment detection by edge drawing (ed). *J. Vis. Commun. Image Represent.* **2017**, *44*, 82–94. [\[CrossRef\]](#)
17. Wen, C.; Liu, P.; Ma, W.; Jian, Z.; Lv, C.; Hong, J.; Shi, X. Edge detection with feature re-extraction deep convolutional neural network. *J. Vis. Commun. Image Represent.* **2018**, *57*, 84–90. [\[CrossRef\]](#)
18. Qu, Z.; Wang, S.Y.; Liu, L.; Zhou, D.Y. Visual cross-image fusion using deep neural networks for image edge detection. *IEEE Access* **2019**, *7*, 57604–57615. [\[CrossRef\]](#)
19. Wibisono, J.K.; Hang, H.M. Traditional Method Inspired Deep Neural Network for Edge Detection. In Proceedings of the 2020 IEEE International Conference on Image Processing (ICIP), Abu Dhabi, United Arab Emirates, 25–28 October 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 678–682.
20. Wang, L.; Shen, Y.; Liu, H.; Guo, Z. An accurate and efficient multi-category edge detection method. *Cogn. Syst. Res.* **2019**, *58*, 160–172. [\[CrossRef\]](#)
21. Fang, T.; Zhang, M.; Fan, Y.; Wu, W.; Gan, H.; She, Q. Developing a feature decoder network with low-to-high hierarchies to improve edge detection. *Multimed. Tools Appl.* **2021**, *80*, 1611–1624. [\[CrossRef\]](#)
22. Le, T.; Duan, Y. REDN: A recursive encoder-decoder network for edge detection. *IEEE Access* **2020**, *8*, 90153–90164. [\[CrossRef\]](#)
23. Soria, X.; Pomboza-Junez, G.; Sappa, A.D. LDC: Lightweight dense CNN for edge detection. *IEEE Access* **2022**, *10*, 68281–68290. [\[CrossRef\]](#)
24. Wibisono, J.K.; Hang, H.M. Fined: Fast Inference Network for Edge Detection. In Proceedings of the 2021 IEEE International Conference on Multimedia and Expo (ICME), Shenzhen, China, 5–9 July 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–6.
25. Liu, H.; Yang, Z.; Zhang, H.; Wang, C. Edge detection with attention: From global view to local focus. *Pattern Recognit. Lett.* **2022**, *154*, 99–109. [\[CrossRef\]](#)
26. Al-Amaren, A.; Ahmad, M.O.; Swamy, M.N.S. RHN: A residual holistic neural network for edge detection. *IEEE Access* **2021**, *9*, 74646–74658. [\[CrossRef\]](#)
27. Ye, Y.; Yi, R.; Cai, Z.; Xu, K. Stedge: Self-training edge detection with multilayer teaching and regularization. *IEEE Trans. Neural Netw. Learn. Syst.* **2023**. *Early Access*. [\[CrossRef\]](#)
28. Elharrouss, O.; Hmamouche, Y.; Idrissi, A.K.; El Khamlichi, B.; El Fallah-Seghrouchni, A. Refined edge detection with cascaded and high-resolution convolutional network. *Pattern Recognit.* **2023**, *138*, 109361. [\[CrossRef\]](#)
29. Wang, Z.; Li, K.; Wang, X.; Lee, A. An Image Edge Detection Algorithm Based on Multi-Feature Fusion. *Comput. Mater. Contin.* **2022**, *73*, 4995–5009. [\[CrossRef\]](#)
30. Soria, X.; Sappa, A.; Humanante, P.; Akbarinia, A. Dense extreme inception network for edge detection. *Pattern Recognit.* **2023**, *139*, 109461. [\[CrossRef\]](#)
31. Poma, X.S.; Riba, E.; Sappa, A. Dense Extreme Inception Network: Towards a Robust CNN Model for Edge Detection. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, Snowmass, CO, USA, 1–5 March 2020; pp. 1923–1932.
32. Xie, S.; Tu, Z. Holistically-Nested Edge Detection. In Proceedings of the IEEE International Conference on Computer Vision, Santiago, Chile, 7–13 December 2015; pp. 1395–1403.
33. Monicka, S.G.; Manimegalai, D.; Karthikeyan, M. Detection of microcracks in silicon solar cells using Otsu-Canny edge detection algorithm. *Renew. Energy Focus* **2022**, *43*, 183–190. [\[CrossRef\]](#)

34. Luque-Chang, A.; Cuevas, E.; Fausto, F.; Zaldivar, D.; Pérez, M. Social spider optimization algorithm: Modifications, applications, and perspectives. *Math. Probl. Eng.* **2018**, *2018*, 6843923. [[CrossRef](#)]
35. Arbelaez, P.; Maire, M.; Fowlkes, C.; Malik, J. Contour detection and hierarchical image segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.* **2010**, *33*, 898–916. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.